Session 17 Homework — Answering Real Questions With SQL

CORE, STRETCH, and CHALLENGE are all required — every one of them must be completed. The three names describe increasing difficulty only, nothing about which parts you submit.

Context

By the end of class, a real, six-table PostgreSQL schema exists in `sql-practice/schema.sql`, matching `projects/b4f-board/specification/DATABASE-PROGRESSION.md`'s own frozen design: `users`, `workspaces`, `workspace_members`, `projects`, `issues`, `comments`. Every real constraint (PRIMARY KEY, UNIQUE, FOREIGN KEY, CHECK) is proven, and every real foreign key's own delete behavior is deliberately chosen and proven. You have written real `SELECT` queries with `WHERE`, `ORDER BY`, and `JOIN` — but only ever to look at rows one at a time, and only ever where every row actually had a match on both sides. Nobody has shown you, live or in teamwork, how to answer a question about a real *group* of rows at once ("how many," "which one is biggest," "per project," "per workspace"), or how to keep a row in the result even when it has no match at all — a real project with zero issues.

Goal

A real, saved query file, `sql-practice/queries.sql`, containing working, proven SQL that answers four real product questions against the schema you already built — the first two using only concepts already shown live, the next two requiring genuinely new concepts you research yourself: `LEFT JOIN`, an **aggregate function**, and `GROUP BY`.

What to build

You already know `JOIN`. Research, then write:

- A query combining `WHERE` and `ORDER BY` to answer a simple, filtered, ordered question.
- To keep a project in the result even when it has zero matching issues, research `LEFT JOIN` — look for the PostgreSQL joined-table type that preserves every row from the left-hand table, even with no match on the right. Combine it with an **aggregate function** and `GROUP BY` — research all three real SQL concepts from the official PostgreSQL documentation; none was shown live or in teamwork. `GROUP BY` collapses many real rows into one row per distinct group; an aggregate function (start with `COUNT`) summarizes each group.
- A query requiring the already-taught `JOIN` and `GROUP BY` to work together in the same statement.
- A short, real, written answer to a real design question (see CHALLENGE), placed directly above the query it explains — the same convention Session 16's own homework already used.

CORE (required)

- `sql-practice/queries.sql` exists, with a query answering: "every real issue in a specific project, ordered by its own key." Use `WHERE` and `ORDER BY`.

- A second query answering: "how many issues does each real project have?" — using `LEFT JOIN`, `COUNT`, and `GROUP BY`. A project with zero issues must still appear in the result, with a real `issue_count` of 0 — a plain `JOIN` would silently drop it; that is exactly why `LEFT JOIN` is required here.

STRETCH (required)

A third query, combining an already-taught `JOIN` with `GROUP BY` / `COUNT` in one statement, answering "how many real members does each workspace have, by name, ranked highest member-count first?"

CHALLENGE (required)

Write a short, direct, written answer, placed directly above a final query, in `sql-practice/queries.sql`:

Why must `workspace_members` be its own real table, rather than, say, a plain array column directly on `workspaces` (for example `member_ids INTEGER[]`)? Give two separate, concrete real reasons, grounded in this exact schema — one about where a member's own `role` could live, one about which real constraint this schema already enforces that an array column could not. Then write one final query: which single real workspace has the most members?

Rules

Allowed: everything already shown live or in teamwork — `CREATE TABLE`, `PRIMARY KEY`, `FOREIGN KEY` / `REFERENCES`, `UNIQUE`, `CHECK`, `INSERT`, `SELECT`, `WHERE`, `ORDER BY`, `JOIN`, plain SQL comments (`--`). Research and use `LEFT JOIN`, `COUNT`, and `GROUP BY` — PostgreSQL's own real documentation on joined tables and on aggregate functions is the right place to start, not a generic tutorial.

Not yet: no other aggregate functions beyond `COUNT` are required (`SUM` / `AVG` / `MIN` / `MAX` exist but are not needed for any real question here); no `HAVING`; no subqueries; no schema changes — `sql-practice/schema.sql` is untouched this homework, only `sql-practice/queries.sql` is new.

Test it

Re-seed a fresh database before testing, exactly in the order the real class session built it (`schema.sql`, then the live-code seed inserts, then the teamwork seed inserts) — several of this session's own live proofs deliberately delete real rows, so a database left over from class will not match the expected results below.

BASH

```
createdb b4f_board_sql_practice_hw   # or drop/recreate your own practice database
psql -d b4f_board_sql_practice_hw -f sql-practice/schema.sql
# re-run the seed INSERT statements from this session's own live-code and teamwork steps, in
order

# then run your own saved queries file end to end
psql -d b4f_board_sql_practice_hw -f sql-practice/queries.sql
```

Done when

- `sql-practice/queries.sql` exists and runs cleanly, end to end, against a freshly re-seeded database, with zero errors.
- The CORE filtered/ordered query returns the correct real rows.
- The CORE aggregate query returns one row per real project, including any project with zero real issues, with a correct real count.
- STRETCH correctly combines `JOIN` and `GROUP BY` in one statement.
- Your CHALLENGE written answer gives two separate, concrete, schema-grounded reasons — not a generic explanation of relational design theory — and the final query correctly names the single real workspace with the most members.

Working time

About 60–80 minutes — the first query reuses concepts already shown live; the genuinely new material (`LEFT JOIN`, aggregate functions, and `GROUP BY`) deserves real research time before writing any SQL.